
Rapport de Projet de Fin d'Année

Filière : Ingénierie Informatique

Ma Santé En Chaîne

*Conception et Développement d'une Plateforme de Santé
Décentralisée basée sur la Blockchain, le Chiffrement Hybride et
le Stockage Distribué IPFS*

Réalisé par :

Yassir Kadouari
Marouane Ismaili
Matine Alkasbiji
Ahmed Amr

Encadré par :

[Nom de l'encadrant]

Année Universitaire : 2024 / 2025

Dédicace

À nos chers parents,

Pour leur amour inconditionnel, leur soutien sans faille et leurs précieux conseils tout au long de notre parcours académique. Vous avez toujours cru en nous et nous avez donné les moyens de réaliser nos ambitions. Ce travail est le fruit de vos sacrifices et de votre dévouement.

À nos familles et amis,

Pour leur présence réconfortante et leurs encouragements constants, vous avez été une source d'inspiration et de motivation. Sans vous, ce projet n'aurait pas vu le jour.

Remerciements

Au terme de ce projet de fin d'année, nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce travail.

Nous adressons nos sincères remerciements à notre encadrant pédagogique pour sa disponibilité, son expertise technique et ses orientations précieuses. Son accompagnement rigoureux nous a permis de surmonter les nombreux défis inhérents au développement d'une application décentralisée (dApp), notamment l'apprentissage du langage Rust, la maîtrise des Smart Contracts ink!, et l'implémentation de protocoles cryptographiques avancés.

Nous remercions également l'ensemble du corps professoral pour la qualité de la formation dispensée, qui nous a dotés des compétences nécessaires pour aborder un projet d'une telle envergure technique.

Enfin, nous exprimons notre reconnaissance aux membres du jury qui ont consacré du temps à l'examen de ce rapport. Leurs retours constructifs contribueront à notre perfectionnement professionnel.

Résumé

Dans un contexte où la numérisation des systèmes de santé s'accélère, la sécurité et la confidentialité des données médicales constituent des enjeux critiques. Les architectures centralisées traditionnelles présentent des vulnérabilités structurelles : points de défaillance uniques (SPOF), risques de falsification documentaire, et absence de souveraineté numérique pour le patient.

Le projet « **Ma Santé En Chaîne** » apporte une réponse technologique innovante en proposant une plateforme 100% décentralisée de gestion des dossiers médicaux. L'architecture repose sur trois piliers fondamentaux :

- Un **Smart Contract ink!** déployé sur un nœud **Substrate**, assurant la traçabilité immuable des actes médicaux et la gestion des droits d'accès (ACL).
- Le réseau **IPFS** (via Pinata) pour le stockage décentralisé des documents médicaux chiffrés.
- Un **chiffrement hybride** combinant AES-256-GCM (symétrique) et Curve25519/XSalsa20/Poly1305 (asymétrique) pour garantir la confidentialité de bout en bout.

L'application frontend, développée en **Next.js 14**, orchestre l'ensemble des interactions entre les différents acteurs du système de santé (Patients, Médecins, Pharmacies, Assurances, Laboratoires, Hôpitaux) au travers d'une authentification par portefeuille cryptographique (Wallet-First).

Mots-Clés : Blockchain, Substrate, ink!, IPFS, Web3, Chiffrement Hybride, AES-GCM, Curve25519, Next.js, Santé Décentralisée, Zero-Knowledge.

Abstract — In an increasingly digitized healthcare environment, the security and confidentiality of medical data have become critical challenges. The "Ma Santé En Chaîne" project proposes a fully decentralized medical records management platform leveraging Web3 technologies : an ink! Smart Contract on Substrate for immutable traceability, IPFS for encrypted off-chain storage, and a hybrid cryptographic envelope (AES-GCM + Curve25519) ensuring end-to-end confidentiality. The Next.js frontend orchestrates secure interactions between healthcare stakeholders through wallet-based authentication.

Keywords : Blockchain, Substrate, ink!, IPFS, Web3, Hybrid Encryption, Decentralized Healthcare.

Glossaire

Terme	Définition
Blockchain	Registre distribué, transparent et immuable, fonctionnant sans organe central de contrôle.
Substrate	Framework modulaire open-source en Rust, développé par Parity Technologies, pour la construction de blockchains personnalisées.
ink!	eDSL (Embedded Domain Specific Language) basé sur Rust pour l'écriture de Smart Contracts exécutables sur Substrate.
IPFS	InterPlanetary File System. Protocole pair-à-pair pour le stockage de fichiers distribué basé sur l'adressage par contenu.
CID	Content Identifier. Identifiant cryptographique unique dérivé du contenu d'un fichier sur IPFS.
AES-GCM	Advanced Encryption Standard – Galois/Counter Mode. Algorithme de chiffrement symétrique authentifié.
Curve25519	Courbe elliptique utilisée pour l'échange de clés Diffie-Hellman (X25519).
XSalsa20-Poly1305	Algorithme de chiffrement authentifié asymétrique utilisé dans les "boîtes" NaCl/TweetNaCl.
SHA-256	Fonction de hachage cryptographique produisant une empreinte de 256 bits.
Wallet	Portefeuille numérique stockant les paires de clés publiques/privées et servant d'identité sur la blockchain.
dApp	Application Décentralisée s'exécutant sur un réseau blockchain.
On-Chain	Données stockées directement sur le registre de la blockchain (immuable, public).
Off-Chain	Données stockées en dehors de la blockchain (IPFS, serveurs locaux).
ACL	Access Control List. Matrice de droits définissant qui peut lire ou modifier une ressource.
Dry Run	Simulation d'une transaction blockchain sans l'exécuter réellement, pour estimer les coûts et vérifier la validité.
Gas	Unité de mesure du coût computationnel d'une transaction sur la blockchain.

Terme	Définition
WebAssembly (Wasm)	Format d'instruction binaire portable, cible de compilation des contrats ink !.

Table des matières

Dédicace	1
Remerciements	2
Résumé	3
Glossaire	4
Introduction générale	10
1 Contexte Général du Projet	11
1.1 Introduction	11
1.2 Présentation du Projet	11
1.2.1 Organisme d'accueil	11
1.2.2 Problématique	11
1.2.3 Objectifs du Projet	12
1.3 Méthodologie de Conduite de Projet	12
1.4 Conclusion	12
2 Analyse et Conception	14
2.1 Introduction	14
2.2 Identification des Acteurs	14
2.3 Spécification des Besoins	15
2.3.1 Besoins Fonctionnels	15
2.3.2 Besoins Non Fonctionnels	16
2.4 Conception du Projet	16
2.4.1 Diagramme de Cas d'Utilisation	16
2.4.2 Diagrammes de Séquence	18
2.4.3 Modélisation des Données On-Chain (Smart Contract ink!)	21
2.4.4 Machine à États	22
2.5 Conclusion	22
3 Technologies et Outils Utilisés	23
3.1 Introduction	23
3.2 Technologies	24
3.2.1 Technologies utilisées pour la couche Blockchain (Back-end décentralisé)	24
3.2.2 Technologies utilisées pour le stockage décentralisé	25
3.2.3 Technologies utilisées pour le développement Frontend	25
3.2.4 Technologies utilisées pour la couche cryptographique	26

3.3	Outils de Développement	26
3.4	Conclusion	26
4	Réalisation et Fonctionnement Détaillé	28
4.1	Introduction	28
4.2	Architecture Technique Globale	28
4.3	Mécanique de Chiffrement Hybride (Zero-Knowledge)	28
4.4	Données On-Chain vs Off-Chain	29
4.5	Pipeline d'Authentification « Wallet-First »	29
4.6	Présentation des Dashboards	30
4.6.1	Accès à l'application	30
4.6.2	Dashboard Médecin	30
4.6.3	Dashboard Pharmacie	31
4.6.4	Dashboard Patient	33
4.6.5	Dashboard Assurance (Insure_Node)	34
4.6.6	Dashboard Laboratoire	35
4.6.7	Dashboard Hôpital	35
4.6.8	Dashboard Administrateur	36
4.7	Défis Techniques Rencontrés et Solutions	37
4.8	Conclusion	37
	Conclusion Générale et Perspectives	38
	Références	39

Table des figures

2.1	Diagramme global des cas d'utilisation de la plateforme	17
2.2	Diagramme de Séquence : Émission et Ancrage d'une Ordonnance	18
2.3	Diagramme de Séquence : Pipeline de Vérification et Délivrance par la Pharmacie	19
2.4	Diagramme de Séquence : Pipeline de Soumission, Audit et Remboursement	21
4.1	Tableau de bord Médecin : Interface de Prescription Décentralisée	31
4.2	Tableau de bord Pharmacie : Vue d'ensemble des ordonnances	32
4.3	Pharmacie : Scan du QR Code de l'ordonnance	32
4.4	Pharmacie : Résultat du scan et vérification d'intégrité blockchain	33
4.5	Tableau de bord Patient : Historique médical et gestion des accès	34
4.6	Tableau de bord Assurance : Moteur d'audit et remboursement	35
4.7	Tableau de bord Laboratoire : Soumission de résultats d'analyses	35
4.8	Tableau de bord Hôpital : Gestion des actes médicaux	36
4.9	Tableau de bord Administrateur : Gouvernance et attribution des rôles . .	36

Liste des tableaux

1.1	Tableau récapitulatif des objectifs du projet	12
2.1	Identification et responsabilités des acteurs du système	15
2.2	Structures de données stockées on-chain dans le Smart Contract	22
3.1	Technologies de la couche Blockchain	24
3.2	Technologies de stockage décentralisé	25
3.3	Technologies Frontend	25
3.4	Technologies cryptographiques	26
3.5	Outils de développement utilisés	26
4.1	Répartition des données entre Blockchain et IPFS	29
4.2	Défis techniques et solutions implémentées	37

Introduction générale

Les systèmes d'information de santé occupent une place centrale dans l'écosystème médical moderne. Cependant, ils sont confrontés à des défis systémiques majeurs qui compromettent leur fiabilité et la confiance des patients. La fragmentation des données entre établissements, le manque d'interopérabilité, et surtout la multiplication des cyberattaques ciblant les infrastructures hospitalières centralisées constituent autant de menaces pour la confidentialité et l'intégrité des dossiers médicaux.

En parallèle, le circuit traditionnel des ordonnances souffre de vulnérabilités exploitables : la falsification documentaire permet des délivrances illégales de substances contrôlées, tandis que les fraudes aux assurances — doubles remboursements, déclarations fictives — engendrent des pertes financières considérables pour l'ensemble du système de santé.

Face à ce constat, le paradigme de la centralisation montre ses limites intrinsèques. La technologie Blockchain, par ses propriétés fondamentales d'immutabilité, de transparence et de décentralisation, offre une alternative architecturale prometteuse pour résoudre ces problématiques.

C'est dans ce contexte qu'est né le projet « **Ma Santé En Chaîne** ». Il s'agit d'une plateforme Web3 complète visant à sécuriser et optimiser l'ensemble du processus médical : de la consultation chez le médecin, à la délivrance du médicament en pharmacie, jusqu'à l'audit et au remboursement par la compagnie d'assurance. La confiance n'est plus déléguée à un administrateur de base de données, mais garantie par des preuves cryptographiques immuables inscrites sur un registre distribué — selon le principe fondateur du Web3 : « *Code is Law* ».

D'un point de vue technique, le projet a fait l'objet d'une migration architecturale radicale : l'élimination totale du serveur d'application classique (Node.js/MongoDB/Express) au profit d'une architecture « Wallet-First » où la logique métier critique s'exécute directement sur la blockchain via des Smart Contracts compilés en WebAssembly.

Ce rapport technique est structuré en quatre chapitres :

- **Chapitre 1** : Contexte général du projet — problématique, objectifs et méthodologie.
- **Chapitre 2** : Analyse et conception — acteurs, cas d'utilisation, diagrammes UML et modélisation des données.
- **Chapitre 3** : Technologies et outils utilisés — l'arsenal technologique déployé.
- **Chapitre 4** : Réalisation et fonctionnement détaillé — architecture technique, pipelines cryptographiques, fonctionnement de chaque dashboard et défis rencontrés.

Chapitre 1

Contexte Général du Projet

1.1 Introduction

Ce premier chapitre pose les fondations du projet « Ma Santé En Chaîne ». Il présente la problématique à laquelle nous répondons, les objectifs techniques et fonctionnels fixés, ainsi que la méthodologie de développement adoptée pour conduire un projet d'une telle complexité technologique.

1.2 Présentation du Projet

1.2.1 Organisme d'accueil

Ce projet a été réalisé dans le cadre de notre formation en Ingénierie Informatique. Il s'inscrit dans une démarche de recherche appliquée visant à explorer les possibilités offertes par les technologies blockchain pour le secteur de la santé.

1.2.2 Problématique

Le secteur médical repose actuellement sur des architectures « Client-Serveur » centralisées qui génèrent des failles structurelles critiques :

1. **Points de Défaillance Uniques (SPOF)** : Un accès non autorisé au serveur central expose l'intégralité des dossiers patients. En 2023, plus de 130 millions de dossiers médicaux ont été compromis à l'échelle mondiale.
2. **Falsification des Actes Médicaux** : Une ordonnance papier ou un PDF peut être falsifié via des outils d'édition grand public, entraînant des délivrances illégales de substances contrôlées ou des doubles remboursements frauduleux.
3. **Absence de Souveraineté Numérique** : Le patient ne possède pas ses données. Elles sont stockées en silos cloisonnés par divers professionnels de santé, sans que le patient ne puisse contrôler finement qui y accède, quand, et pourquoi.
4. **Manque d'Interopérabilité** : Les systèmes propriétaires des hôpitaux, cliniques et laboratoires ne communiquent pas entre eux, fragmentant le parcours de soin du patient.

1.2.3 Objectifs du Projet

Pour pallier ces failles, « Ma Santé En Chaîne » s’est fixé des objectifs techniques ambitieux :

TABLE 1.1 – Tableau récapitulatif des objectifs du projet

Objectif	Description technique
Confidentialité	Les données médicales sont chiffrées sur l’appareil de l’utilisateur (AES-256-GCM). Le réseau IPFS et la blockchain ne reçoivent que des données chiffrées.
Immuabilité	Chaque acte génère une empreinte SHA-256 gravée sur la blockchain. Toute altération du document off-chain invalide instantanément la preuve on-chain.
Contrôle d’Accès	Le Smart Contract gère dynamiquement une matrice de droits (ACL). Seules les adresses Wallet autorisées peuvent interagir avec un dossier.
Désintermédiation	Architecture « Wallet-First » : les acteurs interagissent directement avec le réseau (peer-to-peer), sans serveur intermédiaire.
Traçabilité Complète	Chaque transition d’état (Prescrit → Délivré → Remboursé) est horodatée et signée cryptographiquement sur le registre.

1.3 Méthodologie de Conduite de Projet

La réalisation d’un projet Web3 implique une complexité asymétrique : corriger un bug dans un Smart Contract déployé est extrêmement coûteux (immuabilité du code compilé en WebAssembly). Nous avons donc adopté une approche **Agile / SCRUM** adaptée, organisée en quatre phases itératives :

1. **Sprint 1 — Spécification On-Chain** : Définition stricte des structures de données Rust (`Anchor`, `Claim`), de la machine à état (`Prescribed` → `Delivered` → `Cancelled`), et des règles d’accès avant toute implémentation.
2. **Sprint 2 — Développement Smart Contract** : Écriture du contrat en ink!/Rust, tests unitaires natifs, compilation en WebAssembly et déploiement sur un nœud Substrate local (`substrate-contracts-node`).
3. **Sprint 3 — Intégration Cryptographique** : Développement de la bibliothèque `medicalCrypto.ts` (chiffrement AES-GCM, enveloppes NaCl) et du module `chainContract.ts` (communication RPC avec le nœud Substrate via Polkadot.js).
4. **Sprint 4 — UX/UI et Dashboards** : Construction des sept tableaux de bord spécialisés (Patient, Médecin, Pharmacie, Assurance, Laboratoire, Hôpital, Admin) en Next.js/React/Tailwind CSS.

1.4 Conclusion

Le contexte étant posé, le chapitre suivant détaillera la modélisation conceptuelle du système à travers les diagrammes UML et la spécification des besoins fonctionnels et non

fonctionnels.

Chapitre 2

Analyse et Conception

2.1 Introduction

La modélisation d’une application décentralisée (dApp) exige une rigueur particulière dans la définition des entités et de leurs interactions, car le coût d’une erreur de logique métier sur la blockchain peut être irrécupérable. Ce chapitre présente l’analyse des besoins, l’identification des acteurs, et la conception détaillée du système à travers les diagrammes UML.

2.2 Identification des Acteurs

Chaque acteur interagit avec le système de manière pseudonyme au travers de sa clé publique (adresse Wallet). Le tableau ci-dessous détaille les six rôles identifiés :

TABLE 2.1 – Identification et responsabilités des acteurs du système

Acteur	Rôle et Responsabilités
Patient	Propriétaire souverain de ses ancrages médicaux (Anchors). Consulte son historique, accorde ou révoque des droits d'accès via <code>grant_access</code> / <code>revoke_access</code> , et soumet des requêtes de remboursement (Claims) à son assurance.
Médecin	Autorité médicale prescriptrice. Émet des événements médicaux (Visites, Diagnostics) et des Ordonnances. Chiffre et signe cryptographiquement ces documents avant leur ancrage sur la blockchain.
Pharmacie	Acteur de délivrance. Scanne les ordonnances via QR Code, vérifie mathématiquement leur intégrité (comparaison Hash SHA-256), déchiffre le contenu, et inscrit l'état « Délivré » (<code>DELIVERED</code>) sur la blockchain, verrouillant définitivement la réutilisation.
Assurance	Moteur d'audit financier. Interroge la blockchain pour vérifier la légitimité des Claims, valide l'intégrité cryptographique des actes associés, et émet des statuts de remboursement (<code>APPROVED</code> / <code>REJECTED</code> / <code>REIMBURSED</code>).
Laboratoire / Hôpital	Génère des résultats d'analyses biologiques et des comptes rendus opératoires. Ancre les documents médicaux avec les types <code>LAB_RESULT</code> et <code>OPERATION</code> .
Administrateur	Entité de gouvernance on-chain. Certifie qu'une adresse Wallet correspond à un professionnel de santé accrédité. Gère les attributions de rôles via des ancrages de gouvernance.

2.3 Spécification des Besoins

2.3.1 Besoins Fonctionnels

Au cours de l'analyse, les besoins fonctionnels suivants ont été identifiés :

Besoins relatifs à l'authentification et à l'identité :

- Connexion via un Wallet Web3 (Polkadot.js Extension) — aucun mot de passe.
- Résolution automatique du rôle de l'utilisateur à partir des ancrages de gouvernance stockés sur la blockchain et IPFS.
- Enregistrement de la clé publique de chiffrement sur le contrat intelligent (`register_encryption_k`).

Besoins relatifs au parcours médical :

- Le médecin peut émettre une ordonnance chiffrée, l'ancrer sur la blockchain (`store_anchor`), et spécifier les Wallets autorisés (patient, pharmacie, assurance).
- Le médecin peut créer des événements médicaux (Visites, Résultats d'analyses) avec des types distincts (`VISIT`, `LAB_RESULT`, `OPERATION`).
- La pharmacie peut scanner un QR Code, vérifier l'intégrité via le Hash SHA-256, déchiffrer le contenu et changer le statut en `DELIVERED`.

- Le patient peut consulter l'historique complet de ses actes médicaux et gérer ses accès.

Besoins relatifs aux remboursements :

- Le patient peut soumettre un Claim lié à un `RecordId` délivré (`submit_claim`).
- L'assurance peut auditer les Claims, vérifier les preuves cryptographiques, et émettre une décision (`review_claim`).
- L'assurance peut finaliser un remboursement en ancrant la référence de paiement (`mark_claim_reimbursed`).

2.3.2 Besoins Non Fonctionnels

- **Sécurité** : Toutes les données médicales doivent être chiffrées avant toute transmission. Les clés de déchiffrement sont distribuées via un schéma d'enveloppe hybride (AES-GCM + NaCl Box).
- **Immuabilité** : Les transitions d'état sont strictement régies par le Smart Contract et ne peuvent être contournées par aucun acteur externe.
- **Performance** : Le temps de réponse des lectures on-chain doit rester inférieur à 2 secondes. Un système de cache avec TTL de 12 secondes est implémenté (`CHAIN_READ_CACHE_TTL_M`).
- **Disponibilité** : L'absence de backend centralisé garantit qu'aucun serveur unique ne peut tomber en panne. Les données restent accessibles via les passerelles IPFS et le nœud Blockchain.
- **Compatibilité** : L'application doit fonctionner sur les navigateurs modernes (Chrome, Firefox, Edge) supportant l'extension Polkadot.js.

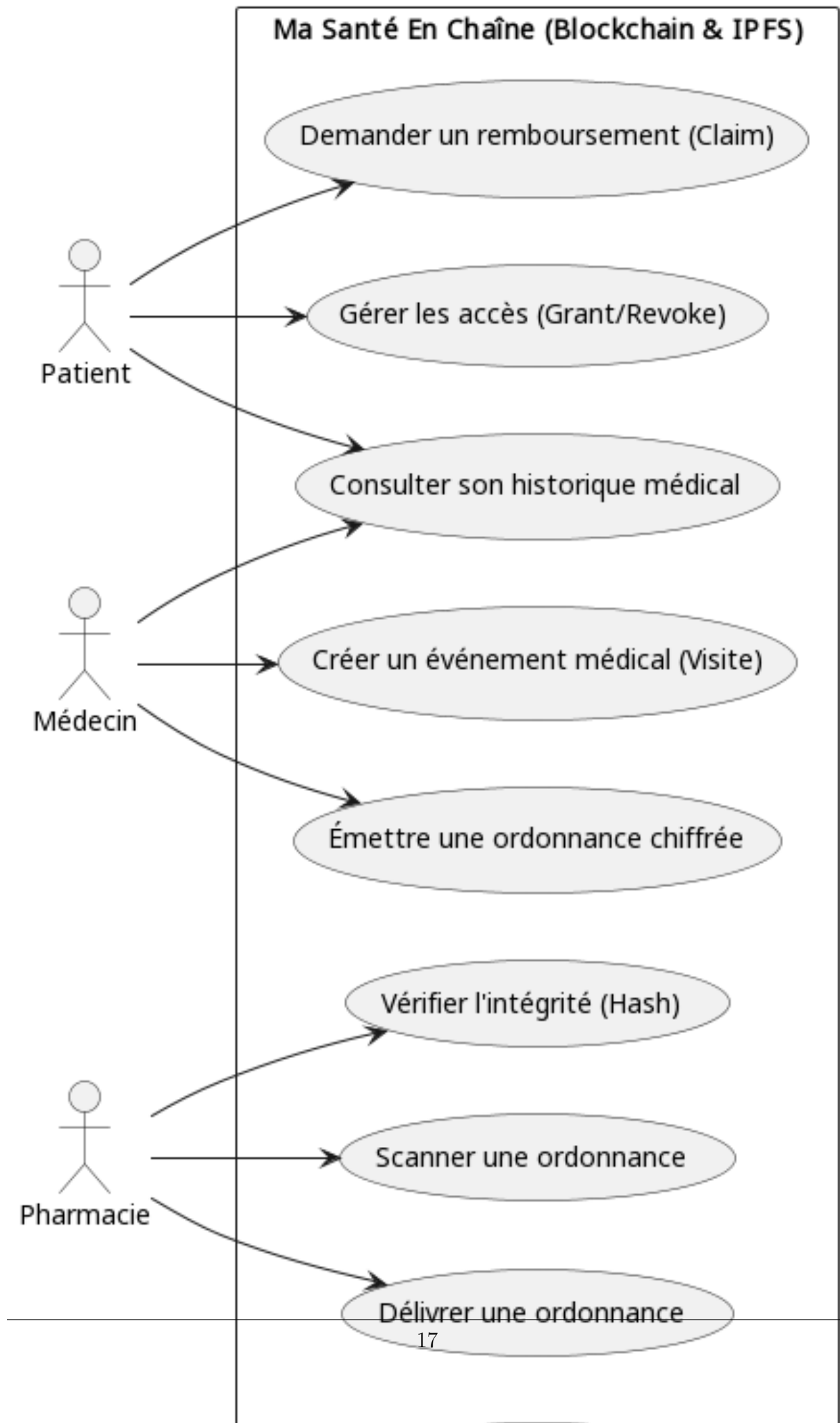
2.4 Conception du Projet

Les diagrammes UML sont des représentations graphiques qui facilitent la compréhension et la communication entre les concepteurs, les développeurs et les parties prenantes d'un projet. Ils permettent de modéliser les différentes perspectives du système.

2.4.1 Diagramme de Cas d'Utilisation

Un diagramme de cas d'utilisation décrit les interactions entre les acteurs et le système. Il met en évidence les différents scénarios possibles où les acteurs interagissent avec la plateforme pour atteindre des objectifs spécifiques.

Le diagramme ci-dessous illustre les interactions et responsabilités des différents acteurs impliqués dans le système « Ma Santé En Chaîne » :



Le diagramme ci-dessus indique que :

Le **Patient** est au centre du système. Il consulte son historique médical immuable, gère finement ses droits d'accès (Grant/Revoke), et soumet des demandes de remboursement (Claims) à son assurance. Ces actions sont directement exécutées sur le Smart Contract via des transactions signées par son Wallet.

Le **Médecin** est l'autorité prescriptrice. Ses responsabilités incluent la création d'événements médicaux (visites, diagnostics) et l'émission d'ordonnances chiffrées. Chaque acte est signé cryptographiquement, chiffré via AES-GCM, envoyé sur IPFS, et ancré sur la blockchain.

La **Pharmacie** agit comme un point de validation critique. Elle scanne les ordonnances, vérifie mathématiquement l'intégrité des documents via la comparaison des Hashs SHA-256, et verrouille définitivement l'ordonnance sur le registre en changeant son statut en DELIVERED.

L'**Assurance** est le moteur d'audit. Elle vérifie les preuves cryptographiques associées aux Claims et émet des décisions de remboursement traçables sur la blockchain.

2.4.2 Diagrammes de Séquence

Un diagramme de séquence décrit comment les objets et les composants d'un système interagissent entre eux au fil du temps. Il met en évidence l'ordre chronologique des messages échangés et montre comment les interactions se déroulent étape par étape. Ce type de diagramme est essentiel pour comprendre le comportement dynamique du système et analyser les flux de communication.

Diagramme de Séquence : Émission d'une Ordonnance

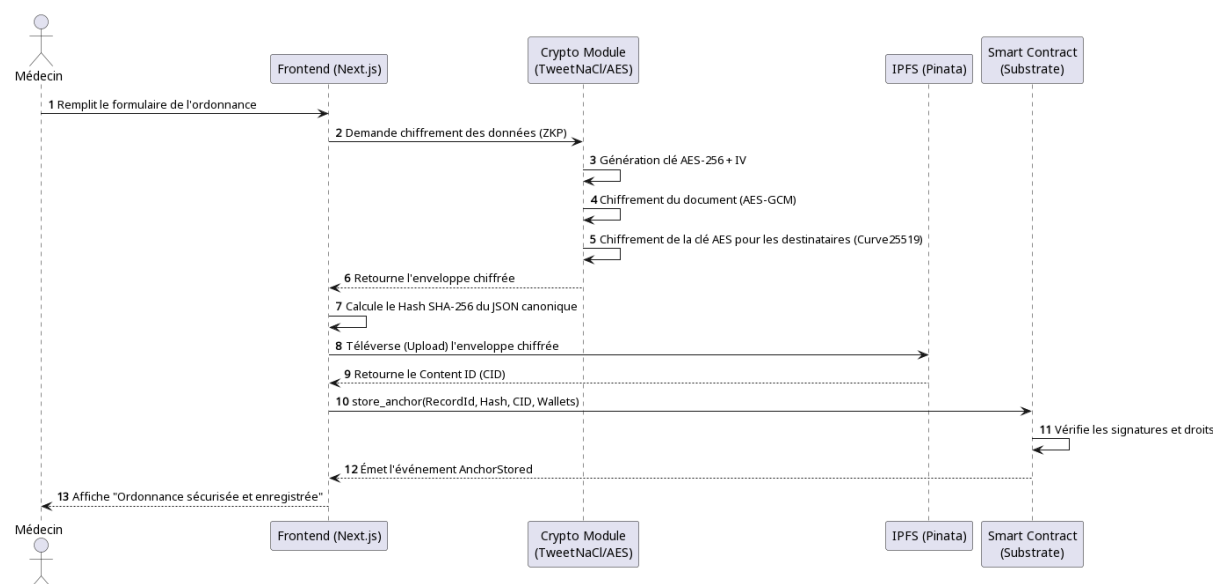


FIGURE 2.2 – Diagramme de Séquence : Émission et Ancrage d'une Ordonnance

Le diagramme ci-dessus indique :

1. Le médecin remplit le formulaire de prescription dans le Frontend Next.js.
2. Le module cryptographique génère une clé AES-256 aléatoire et un IV de 12 octets.

3. Le document JSON est canonisé (tri alphabétique des clés) puis chiffré en AES-GCM.
4. La clé AES est chiffrée pour chaque destinataire (patient, pharmacie) via des boîtes NaCl (Curve25519).
5. L'enveloppe chiffrée est téléversée sur IPFS/Pinata, générant un CID unique.
6. Le Frontend forge une transaction `store_anchor` contenant le RecordId, le Hash SHA-256, le CID, et les Wallets autorisés.
7. Le Smart Contract vérifie la signature et émet l'événement `AnchorStored`.

Diagramme de Séquence : Vérification et Délivrance Pharmacie

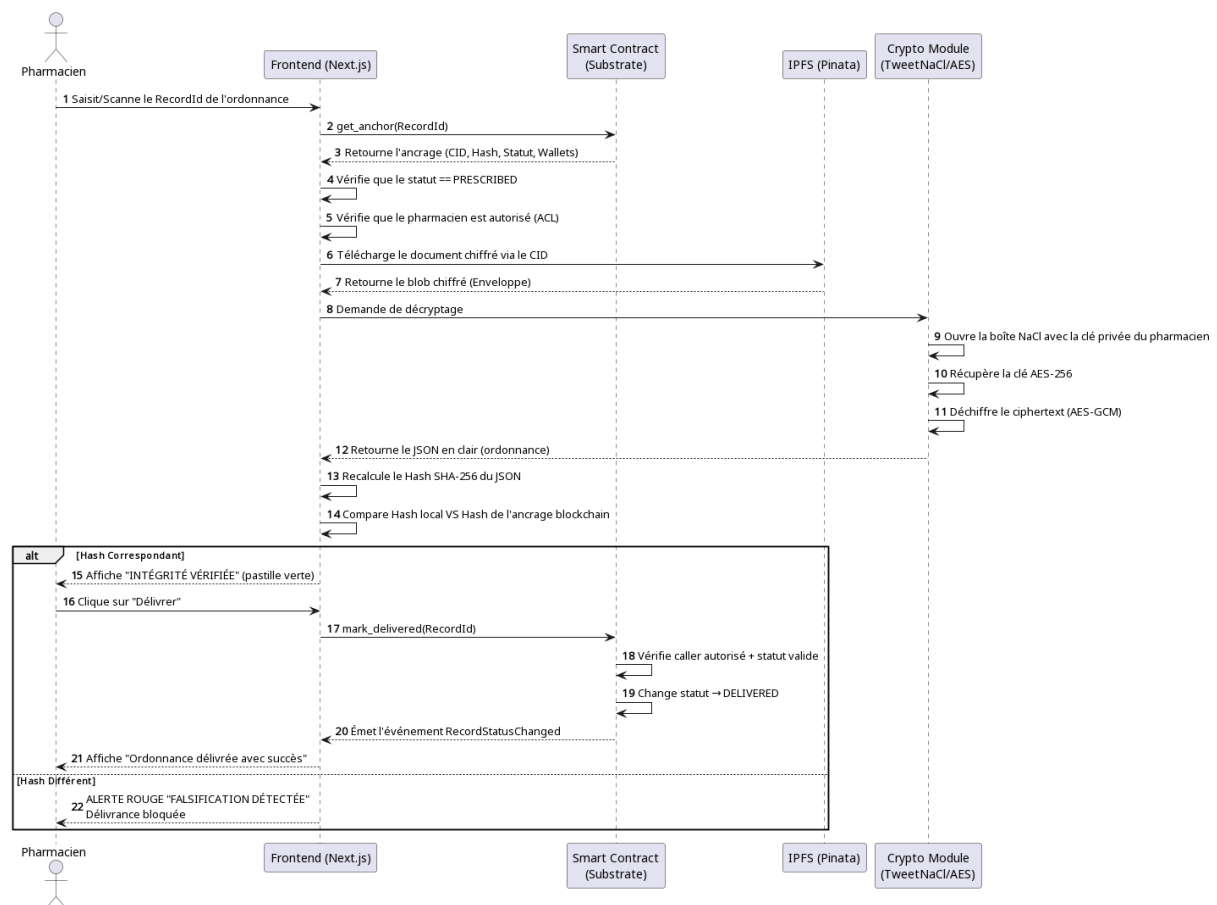


FIGURE 2.3 – Diagramme de Séquence : Pipeline de Vérification et Délivrance par la Pharmacie

Le diagramme ci-dessus indique :

1. Le pharmacien saisit ou scanne le RecordId.
2. Le système interroge le Smart Contract (`get_anchor`) pour récupérer le CID, le Hash, le statut et les Wallets autorisés.
3. Le Frontend vérifie que le statut est `PRESCRIBED` et que le pharmacien est dans l'ACL.
4. Le document chiffré est téléchargé depuis IPFS via le CID.

5. La boîte NaCl du pharmacien est ouverte avec sa clé privée pour récupérer la clé AES.
6. Le ciphertext est déchiffré en AES-GCM, reconstituant le JSON de l'ordonnance.
7. **Étape critique** : Le Hash SHA-256 est recalculé localement et comparé avec celui stocké on-chain.
8. Si les hashes correspondent : intégrité vérifiée, la délivrance est autorisée. Si les hashes divergent : **ALERTE FALSIFICATION**, la délivrance est bloquée.

Diagramme de Séquence : Pipeline Assurance (Claims)

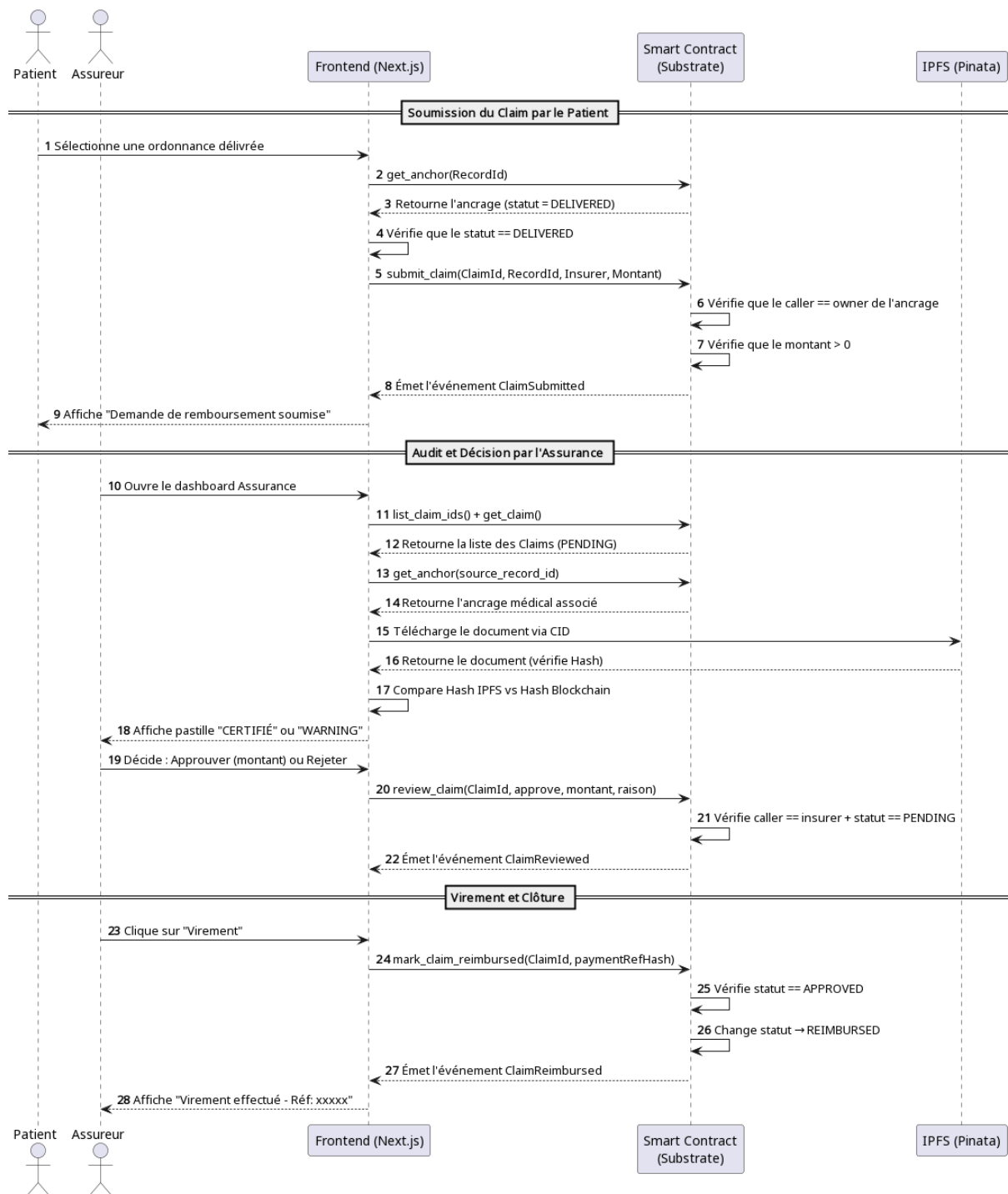


FIGURE 2.4 – Diagramme de Séquence : Pipeline de Soumission, Audit et Remboursement

2.4.3 Modélisation des Données On-Chain (Smart Contract ink !)

Contrairement aux bases de données relationnelles (SQL) ou documentaires (MongoDB), le stockage on-chain est extrêmement coûteux. Nous avons donc conçu des structures de données minimalistes en Rust :

TABLE 2.2 – Structures de données stockées on-chain dans le Smart Contract

Structure	Champs principaux
Anchor	kind (Prescription/Visit/LabResult/Operation), cid (bytes IPFS), data_hash (SHA-256 32 bytes), owner, doctor, pharmacy?, insurer?, status (Prescribed/Delivered/Cancelled), created_at, updated_at
Claim	source_record_id, claimant, insurer, amount_requested, amount_approved?, status (Pending/Approved/Rejected/Reimbursed), reason_hash?, payment_ref_hash?, timestamps
Mappings	records: Mapping<RecordId, Anchor>, claims: Mapping<ClaimId, Claim>, access: Mapping<(RecordId, AccountId), bool>, encryption_keys: Mapping<AccountId, [u8;32]>

2.4.4 Machine à États

Les transitions d'état sont strictement contrôlées par le Smart Contract. Aucune transition invalide ne peut être exécutée :

Cycle de vie d'une Ordonnance :

- PRESCRIBED → DELIVERED (par la pharmacie autorisée)
- PRESCRIBED → CANCELLED (par le médecin ou le patient)
- DELIVERED → toute transition : **BLOQUÉE**
- CANCELLED → toute transition : **BLOQUÉE**

Cycle de vie d'un Claim :

- PENDING → APPROVED / REJECTED (par l'assureur)
- APPROVED → REIMBURSED (par l'assureur après virement)
- REJECTED → toute transition : **BLOQUÉE**
- REIMBURSED → toute transition : **BLOQUÉE**

2.5 Conclusion

Ce chapitre d'analyse et conception nous a permis de définir avec précision les bases de notre système décentralisé. La spécification des acteurs, des cas d'utilisation et la création des diagrammes UML établissent une compréhension claire des fonctionnalités attendues et des interactions sécurisées entre les acteurs et le Smart Contract. Ces éléments constituent le socle de la phase de réalisation détaillée dans les chapitres suivants.

Chapitre 3

Technologies et Outils Utilisés

3.1 Introduction

Dans ce chapitre, nous détaillons les différentes technologies et outils utilisés dans le développement de « Ma Santé En Chaîne ». Chaque technologie a été sélectionnée en fonction de ses avantages spécifiques pour la construction d'une application décentralisée sécurisée et performante.

3.2 Technologies

3.2.1 Technologies utilisées pour la couche Blockchain (Back-end décentralisé)

TABLE 3.1 – Technologies de la couche Blockchain

Technologie	Description et rôle dans le projet
Rust	Langage de programmation système développé par Mozilla, offrant des garanties de sécurité mémoire sans ramasse-miettes (garbage collector). Dans notre projet, Rust est utilisé pour l'écriture du Smart Contract, bénéficiant de son système de typage strict qui élimine par design des classes entières de vulnérabilités (buffer overflow, data races).
Substrate	Framework modulaire open-source développé par Parity Technologies (créateurs de Polkadot). Il permet de construire des blockchains personnalisées avec un consensus configurable. Nous utilisons <code>substrate-contracts-node</code> comme environnement d'exécution local avec stockage persistant dans le dossier <code>blockchain-data/</code> .
ink !	eDSL (Embedded Domain Specific Language) de l'écosystème Polkadot pour l'écriture de Smart Contracts. Contrairement à Solidity (Ethereum), ink! est compilé en WebAssembly (Wasm) natif, offrant des performances supérieures et la sécurité de typage de Rust. Les macros <code>#[ink::contract]</code> et <code>#[ink(message)]</code> simplifient la déclaration des fonctions publiques du contrat.
WebAssembly (Wasm)	Format d'instruction binaire portable servant de cible de compilation pour les contrats ink!. Le fichier <code>.wasm</code> généré est déployé sur le nœud Substrate et exécuté dans un environnement sandboxé et sécurisé.

3.2.2 Technologies utilisées pour le stockage décentralisé

TABLE 3.2 – Technologies de stockage décentralisé

Technologie	Description et rôle dans le projet
IPFS	InterPlanetary File System. Au lieu de localiser un fichier par son emplacement physique (URL), IPFS le localise par son contenu (Content Addressing) via son CID. Si un seul octet du fichier médical change, le CID change complètement, garantissant l'intégrité par design.
Pinata	Service de « pinning » IPFS garantissant que nos fichiers restent en permanence disponibles et répliqués sur le réseau mondial, même si le client d'origine se déconnecte. Pinata expose une API REST sécurisée par JWT pour l'upload et le téléchargement de fichiers.

3.2.3 Technologies utilisées pour le développement Frontend

TABLE 3.3 – Technologies Frontend

Technologie	Description et rôle dans le projet
Next.js 14	Framework React de référence avec App Router. Il sert de moteur d'orchestration global. Bien qu'il n'y ait pas de base de données backend traditionnelle, les API Routes (Edge) servent de proxys légers pour contourner les restrictions CORS des passerelles IPFS.
React 18	Bibliothèque JavaScript pour la construction d'interfaces utilisateur dynamiques et réactives. Les composants React gèrent les états complexes de chaque dashboard (chargement, transactions en cours, vérification d'intégrité).
Tailwind CSS	Framework utilitaire CSS permettant de concevoir rapidement des interfaces modernes. Notre UI adopte un thème « Dark Mode » avec des effets de Glassmorphism et une typographie monospace évoquant l'univers blockchain.
TypeScript	Sur-ensemble typé de JavaScript. L'ensemble du code frontend est écrit en TypeScript, apportant une vérification de types à la compilation et réduisant les erreurs à l'exécution.

3.2.4 Technologies utilisées pour la couche cryptographique

TABLE 3.4 – Technologies cryptographiques

Technologie	Description et rôle dans le projet
Polkadot.js API	Interface de programmation permettant au navigateur de communiquer via WebSockets (RPC) avec le nœud Substrate. Elle gère l'estimation des frais de Gas, l'injection de la signature utilisateur, et le décodage des événements du contrat.
TweetNaCl.js	Implémentation JavaScript audité de la bibliothèque cryptographique NaCl (Networking and Cryptography library). Elle fournit les primitives pour le chiffrement asymétrique Curve25519/XSalsa20/Poly1305 utilisées dans notre schéma d'enveloppe hybride.
Web Crypto API	API native du navigateur pour les opérations cryptographiques (AES-GCM, SHA-256, PBKDF2). Disponible uniquement dans un contexte sécurisé (HTTPS ou localhost).

3.3 Outils de Développement

TABLE 3.5 – Outils de développement utilisés

Outil	Utilisation dans le projet
VS Code	Éditeur de code principal avec extensions Rust-Analyzer et ink! pour le développement du Smart Contract, et ESLint/Prettier pour le code TypeScript.
cargo-contract	CLI officiel pour compiler, tester et déployer les contrats ink! sur Substrate. Commande de build : <code>cargo contract build -release</code> .
Polkadot.js Apps	Interface web pour interagir manuellement avec le nœud Substrate, inspecter les blocs, et debugger les transactions.
Git / GitHub	Système de contrôle de version distribué pour la gestion du code source et la collaboration d'équipe.
PlantUML	Outil de génération de diagrammes UML à partir de descriptions textuelles, utilisé pour produire les diagrammes de ce rapport.

3.4 Conclusion

Ce chapitre a présenté l'arsenal technologique complet déployé pour la réalisation de « Ma Santé En Chaîne ». La combinaison de Rust/ink! pour la sécurité du contrat, IPFS pour le stockage distribué, et Next.js/React pour l'expérience utilisateur constitue

l'état de l'art du développement d'applications décentralisées (dApps). Le chapitre suivant détaillera la réalisation technique et le fonctionnement interne de chaque composant.

Chapitre 4

Réalisation et Fonctionnement Détaillé

4.1 Introduction

Ce chapitre, cœur du rapport, plonge dans l'implémentation technique de « Ma Santé En Chaîne ». Il détaille l'architecture Web3 déployée, les pipelines cryptographiques internes, le fonctionnement de chaque tableau de bord, et les défis techniques rencontrés.

4.2 Architecture Technique Globale

L'architecture du projet repose sur trois couches distinctes, sans aucun serveur backend traditionnel :

1. **Couche Présentation (Next.js)** : Interface utilisateur React avec sept dashboards spécialisés. Orchestre les appels cryptographiques et les transactions blockchain.
2. **Couche Logique Métier (Smart Contract ink!)** : Contrat intelligent déployé sur Substrate. Gère les ancres (**Anchor**), les claims, les transitions d'état et la matrice ACL.
3. **Couche Stockage (IPFS/Pinata)** : Héberge les documents médicaux chiffrés. Le contrat ne stocke que les pointeurs (CID) et les empreintes (Hash SHA-256).

4.3 Mécanique de Chiffrement Hybride (Zero-Knowledge)

Le plus grand défi technique est le respect du secret médical sur un réseau public. Le module `medicalCrypto.ts` implémente une architecture d'enveloppe hybride (`msce-hybrid-aesgcm-v2`).

Processus de chiffrement détaillé :

1. **Génération de la clé symétrique** : Le navigateur génère une clé AES-256 aléatoire et un vecteur d'initialisation (IV) de 12 octets via `crypto.getRandomValues()`.
2. **Canonisation JSON** : L'objet JSON est canonisé (tri alphabétique récursif des clés) pour garantir le déterminisme mathématique du hachage, indépendamment de l'ordre d'insertion des propriétés.
3. **Chiffrement AES-GCM** : Le JSON canonisé est converti en octets puis chiffré via `crypto.subtle.encrypt()` avec l'algorithme AES-GCM (Galois/Counter Mode), produisant le *ciphertext* et un tag d'authentification intégré.

4. **Récupération des clés publiques** : Le système interroge le Smart Contract via `encryption_key_of(walletAddress)` pour obtenir les clés publiques Curve25519 de chaque destinataire.
5. **Enveloppes NaCl** : Pour chaque destinataire, la clé AES est chiffrée dans une « boîte » NaCl utilisant l'échange de clés X25519, le chiffrement XSalsa20 et le MAC Poly1305. Un nonce éphémère unique est généré par boîte.
6. **Construction de l'enveloppe finale** : L'objet JSON envoyé sur IPFS contient : `version`, `algorithm`, `ivB64`, `ciphertextB64`, `payloadHashHex`, et un tableau `encryptedKeys []` (une entrée par destinataire avec `walletAddress`, `senderPublicKeyHex`, `nonceB64`, `encryptedDataKeyB64`).

Le **décryptage** suit le chemin inverse : le destinataire ouvre sa boîte NaCl avec sa clé privée, récupère la clé AES en RAM, déchiffre le ciphertext, et accède au contenu médical. À aucun moment la clé privée ou le document en clair ne quitte le navigateur.

4.4 Données On-Chain vs Off-Chain

TABLE 4.1 – Répartition des données entre Blockchain et IPFS

Stocké sur la Blockchain (On-Chain)	Stocké sur IPFS (Off-Chain)
Identifiants (<code>RecordId</code> , <code>ClaimId</code>)	Ordonnances chiffrées (<code>schema msce-prescription-v2</code>)
Empreintes SHA-256 (32 bytes)	Événements médicaux (visites, analyses)
Pointeurs CID vers IPFS	Tickets de délivrance pharmacie
Statuts (<code>Prescribed</code> , <code>Delivered...</code>)	Documents gouvernance (rôles, identités)
Wallets autorisés (<code>owner</code> , <code>doctor...</code>)	Profils utilisateur
Clés publiques de chiffrement (32 bytes)	PDF encodés en base64 dans JSON
Horodatages de création/modification	—

4.5 Pipeline d'Authentification « Wallet-First »

Le processus de connexion s'affranchit totalement du couple classique email/mot de passe :

1. L'utilisateur clique sur « Se Connecter ». L'extension Polkadot.js du navigateur intercepte la demande.
2. Le module `wallet.ts` appelle `getExtension().enable("Ma Sante en Chaine")` pour obtenir l'accès aux comptes.
3. Le premier compte Wallet disponible est sélectionné. Son adresse publique devient l'identité de l'utilisateur.
4. Le système interroge les ancrages de gouvernance sur la blockchain et IPFS (`resolveWalletRoleOn`) pour déterminer le rôle de cette adresse (Médecin, Pharmacien, Patient, etc.).

5. Une session locale est créée et stockée dans `localStorage` avec le token JWT simulé, le rôle résolu et les métadonnées d'identité.
6. L'utilisateur est redirigé vers le dashboard correspondant à son rôle.

4.6 Présentation des Dashboards

4.6.1 Accès à l'application

La page d'authentification est une interface sécurisée Wallet-First qui vérifie l'identité de l'utilisateur via sa signature cryptographique. Contrairement aux systèmes traditionnels, aucun mot de passe n'est requis. L'identité est prouvée mathématiquement par la possession de la clé privée associée au portefeuille.

4.6.2 Dashboard Médecin

Le tableau de bord du médecin est le point d'entrée principal pour la création d'actes médicaux.

Fonctionnement en arrière-plan lors d'une prescription :

1. Les données du formulaire (médicaments, posologie, instructions) sont agrégées en un objet JSON.
2. Le pipeline de chiffrement hybride est déclenché (`encryptMedicalPayloadOrPlain`).
3. Si les clés publiques des destinataires sont disponibles on-chain, le chiffrement AES-GCM + NaCl est appliqué. Sinon, un mode dégradé (`PLAIN_IPFS_FALLBACK`) est utilisé avec un avertissement.
4. L'enveloppe chiffrée est envoyée à l'API Pinata (`uploadJsonToIpfs`) avec un mécanisme de retry exponentiel (`withUploadRetry`).
5. Le CID retourné par Pinata est utilisé pour forger la transaction `store_anchor` sur le Smart Contract, incluant le `RecordId` (hash Blake2 de la clé logique), le Hash SHA-256 du payload, et les adresses des Wallets autorisés.
6. Un Dry Run (simulation) est exécuté avant la signature pour vérifier que le contrat ne rejettera pas la transaction.
7. L'utilisateur signe la transaction via l'extension Polkadot.js.
8. Une fois le bloc miné, l'interface est notifiée via WebSocket et l'ordonnance apparaît dans la liste.

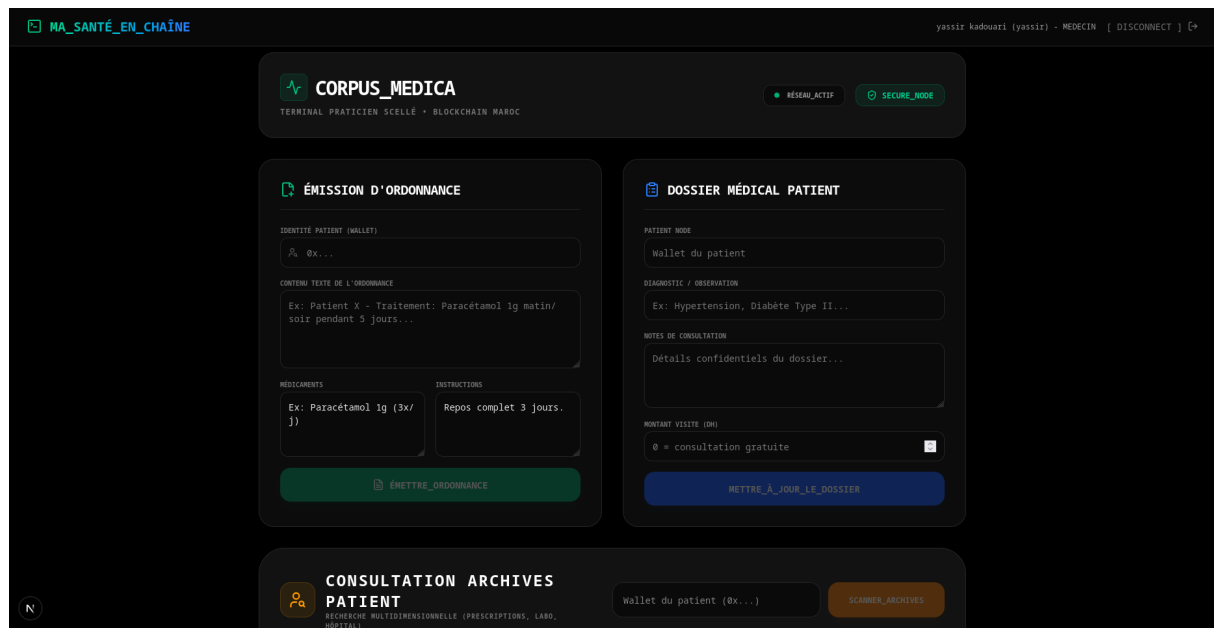


FIGURE 4.1 – Tableau de bord Médecin : Interface de Prescription Décentralisée

4.6.3 Dashboard Pharmacie

L'interface de la pharmacie est conçue comme un scanner et un point de validation critique de la chaîne de confiance.

Mécanique de vérification et délivrance :

1. Le pharmacien saisit ou scanne le QR Code contenant le RecordId. Le système normalise l'entrée via `normalizeAnchorLookupInput()` qui supporte les URLs, les préfixes `msc:prescription:`, et les hashes hexadécimaux.
2. Le Smart Contract est interrogé (`get_anchor`) : l'ordonnance est-elle valide (`Prescribed`) ? Le pharmacien est-il dans l'ACL ?
3. Le document chiffré est téléchargé depuis IPFS via le CID.
4. La boîte NaCl du pharmacien est ouverte, la clé AES est récupérée, et le document JSON est reconstitué.
5. **Étape critique** : Le Hash SHA-256 est recalculé localement (`bodyDigest`) et comparé strictement avec celui stocké on-chain (`verifyHashOnChain`). Si un attaquant modifie le fichier sur IPFS, les hashes divergeront et l'interface affichera une alerte « **FALSIFICATION DÉTECTÉE** », bloquant la délivrance.
6. Si tout est conforme, le pharmacien clique sur « Délivrer ». La transaction `mark_delivered` est forgée, verrouillant définitivement l'ordonnance.
7. Un ticket de délivrance (`receipt:`) est créé et ancré séparément, contenant le montant total.

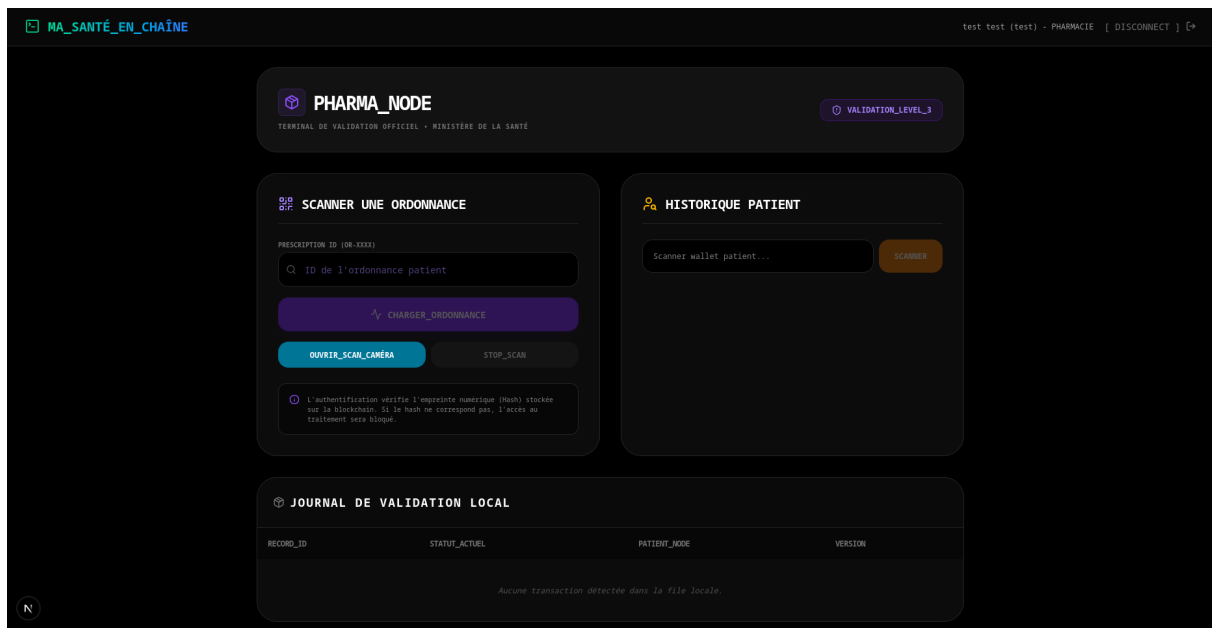


FIGURE 4.2 – Tableau de bord Pharmacie : Vue d’ensemble des ordonnances

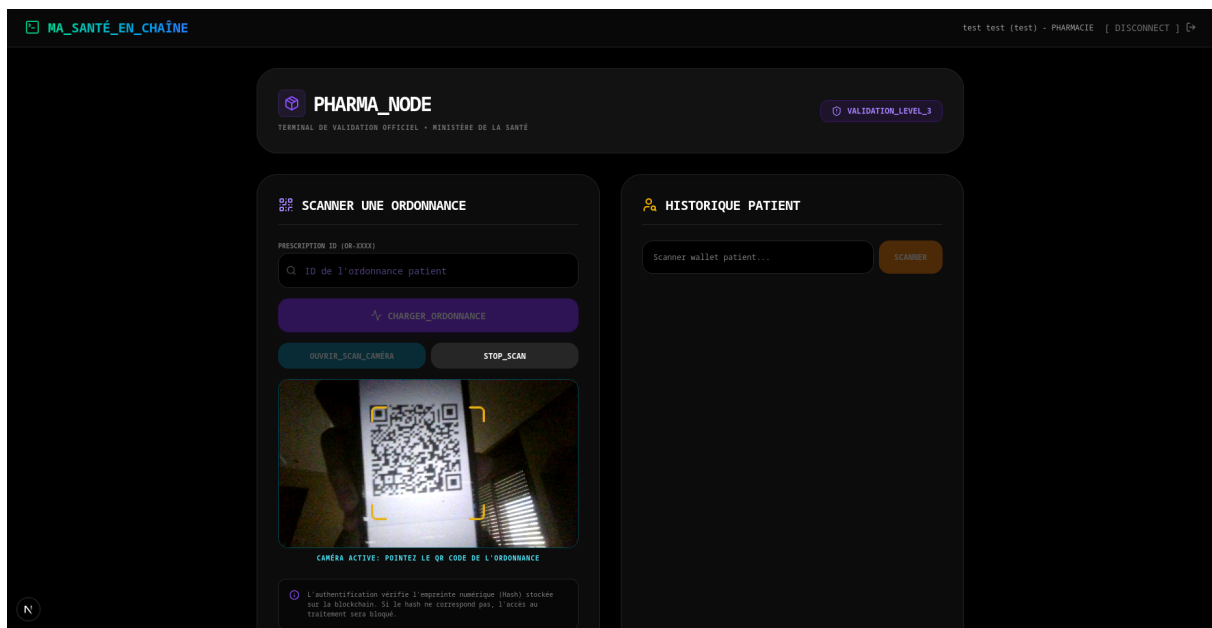


FIGURE 4.3 – Pharmacie : Scan du QR Code de l’ordonnance

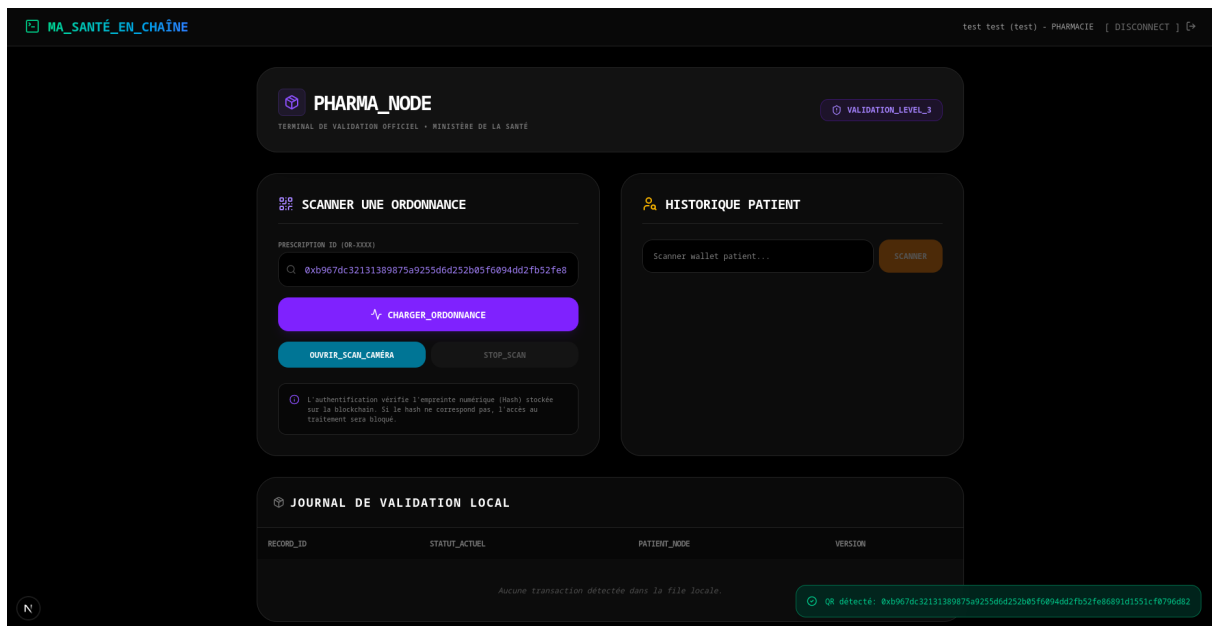


FIGURE 4.4 – Pharmacie : Résultat du scan et vérification d'intégrité blockchain

4.6.4 Dashboard Patient

Le patient dispose d'une vue souveraine sur l'ensemble de son historique médical immuable.

Fonctionnalités principales :

- Consultation de toutes les ordonnances (en cours et délivrées) liées à son Wallet.
- Gestion fine des droits d'accès : le patient peut accorder (**grant_access**) ou révoquer (**revoke_access**) l'accès de n'importe quel praticien à ses dossiers.
- Soumission de demandes de remboursement (Claims) : le patient sélectionne une ordonnance délivrée, spécifie le montant et l'assureur, et la transaction **submit_claim** est envoyée au contrat.
- Configuration du profil médical : médecin traitant, groupe sanguin, pathologies, région.

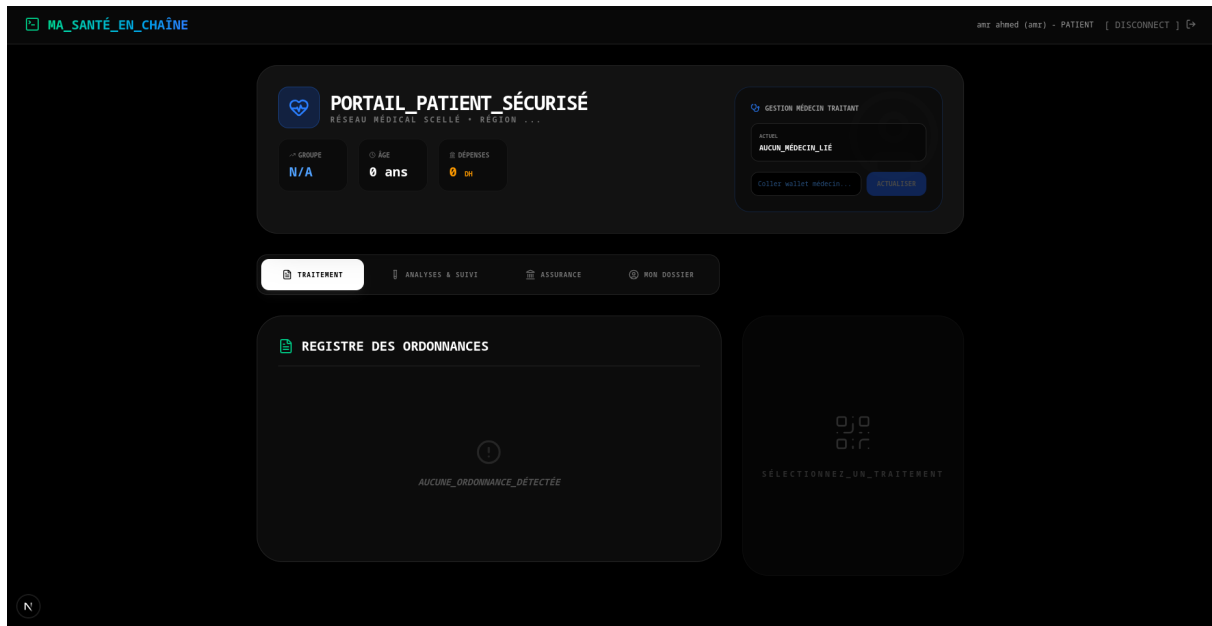


FIGURE 4.5 – Tableau de bord Patient : Historique médical et gestion des accès

4.6.5 Dashboard Assurance (Insure_Node)

Le dashboard de l'assurance est un véritable moteur d'audit cryptographique.

Fonctionnement du moteur de remboursement :

1. Le système liste les Claims via `listClaimsFromChain()` et filtre ceux adressés au Wallet de l'assureur connecté.
2. Pour chaque Claim, le système récupère l'ancrage source (`get_anchor(source_record_id)`), vérifie son statut (`DELIVERED`), et valide l'intégrité du document IPFS en comparant les hashes.
3. Une pastille visuelle « CERTIFIÉ » (verte) ou « WARNING » (rouge) indique le résultat de la vérification.
4. L'agent d'assurance peut approuver (avec montant ajusté et justification) ou rejeter la demande via `review_claim`.
5. Après le virement bancaire externe, l'agent clôture le cycle avec `mark_claim_reimbursed`, ancrant la référence de paiement (`paymentRefHash`) sur la blockchain.

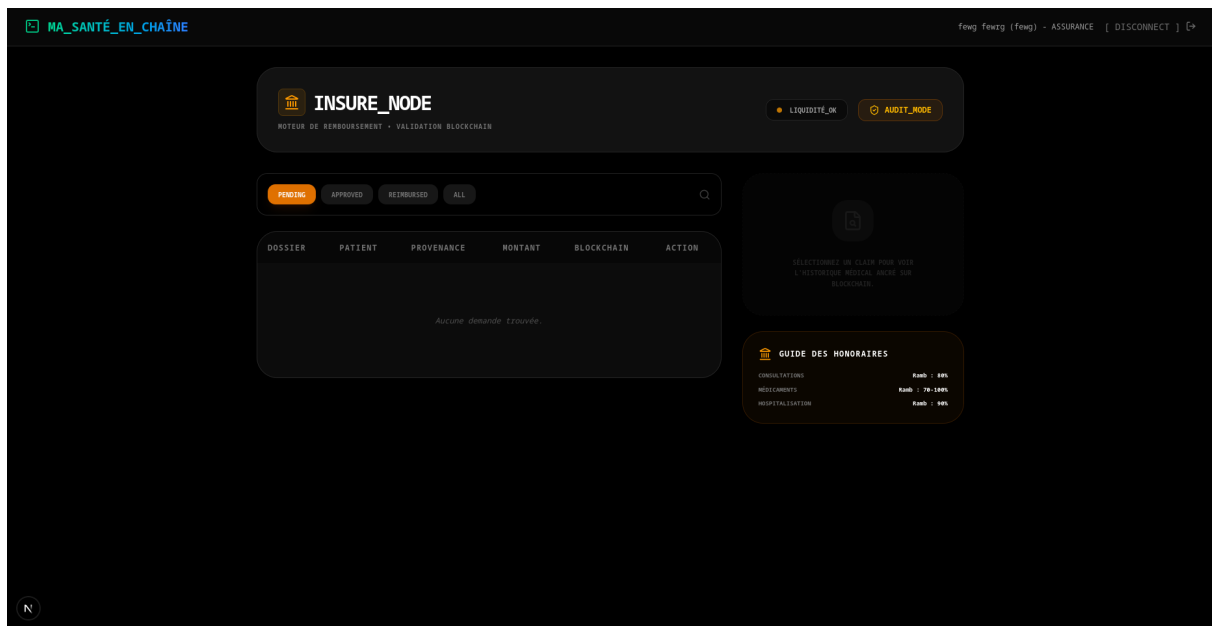


FIGURE 4.6 – Tableau de bord Assurance : Moteur d’audit et remboursement

4.6.6 Dashboard Laboratoire

Le laboratoire peut soumettre des résultats d’analyses biologiques. Chaque résultat est ancré avec le type LAB_RESULT et chiffré pour le patient et le médecin prescripteur.

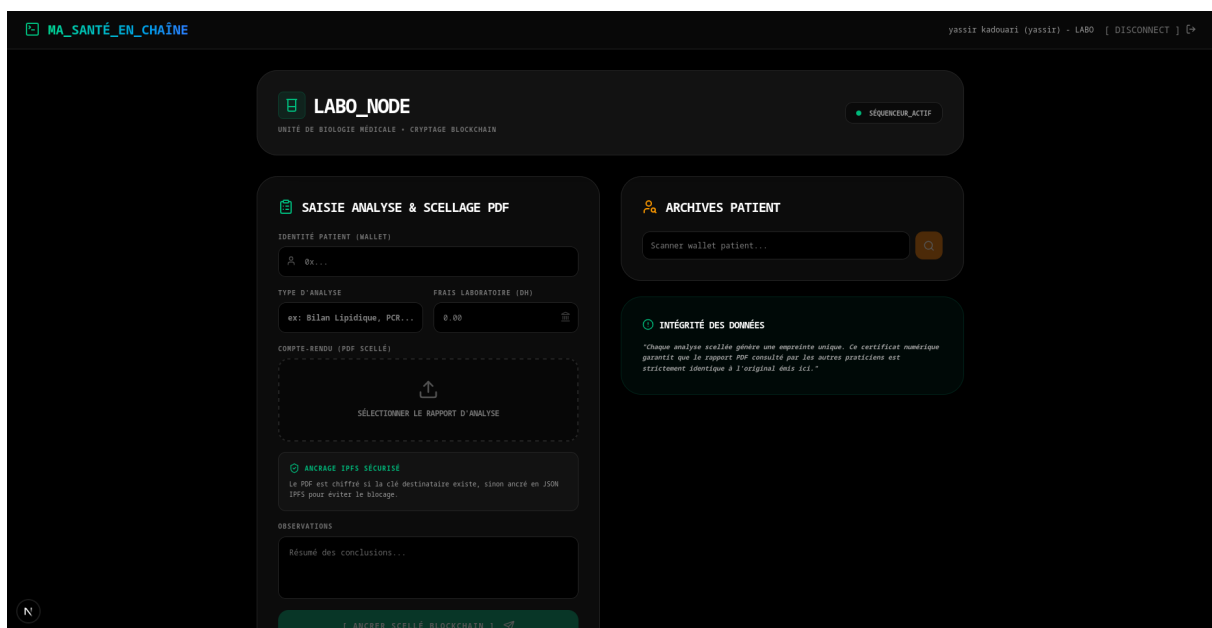


FIGURE 4.7 – Tableau de bord Laboratoire : Soumission de résultats d’analyses

4.6.7 Dashboard Hôpital

L’hôpital peut créer des événements médicaux de type OPERATION et VISIT, incluant des documents PDF encodés en base64 dans le JSON chiffré.

MA_SANTÉ_EN_CHAÎNE matine kezbi (matine) - HOPITAL [DISCONNECT] [+]

ENREGISTREMENT ACTE & SCELLAGE PDF

IDENTITE PATIENT (WALLET)

ex...

TYPE D'ACTE **ADMISSION** DÉPARTEMENT **ex: Cardiologie, Réanimation**

COMPTE-RENDU HOSPITALIER (PDF)

TÉLÉVERSER LE RAPPORT SCELLÉ

ANCRAGE IPFS SÉCURISÉ

Le PDF est chiffré si la clé destination existe, sinon ancré en JSON IPFS pour éviter le blocage.

DÉTAILS DE L'ACTE

Motif de l'intervention...

FACTURATION HONORAIRES (DM)

0.00

[VALIDER_ET_SCELLER_L_ACTE]

ARCHIVES MÉDICALES

Saisir wallet patient...

AUDIT BLOCKCHAIN

"Tous les actes hospitaliers enregistrés ici bénéficient d'une preuve d'intégrité via blockchain. Cela garantit la traçabilité des interventions et facilite les procédures de remboursement."

FIGURE 4.8 – Tableau de bord Hôpital : Gestion des actes médicaux

4.6.8 Dashboard Administrateur

L'administrateur est l'entité de gouvernance. Il attribue les rôles on-chain aux professionnels de santé en créant des ancres de gouvernance sur IPFS, liés aux familles de RecordId `mongo:walletroles:*`.

MA_SANTÉ_EN_CHAÎNE yossir Radouazi (yossir) - ADMIN [DISCONNECT] [+]

GLOBAL_ADMIN_CONSOLE

SECTEUR : RAKHIZEN GLOBAL

AUTH_LEVEL: Lvl_8_GLOBAL

ASSIGNATION DE RÔLES

Adresse Wallet (Polkadot / Web3) **PATIENT** **TANGER-TETOUAN-AL HOCEIMA**

[ASSIGNER_RÔLE] [RÉVOQUER]

REGISTRE DES UTILISATEURS

affichage de tous les secteurs

WALLET	RÔLES	RÉGION	IDENTITÉ & DÉTAILS	ACTIONS_VÉRIFICATION
5GnFLbW...w8Kw9guJ	ASSURANCE	Global	Utilisateur Wallet De: 1000-01-01 sahan sahan	[ACTIF]
SFDukpW...qV0mQJ5	LABO	TANGER-TETOUAN-AL HOCEIMA	Utilisateur Wallet De: 1000-01-01 moul moul	[ACTIF]
SEPz4Mw...e5y5mwb	HOPITAL	Global	Utilisateur Wallet De: 1000-01-01 gagae gagae	[ACTIF]
SG1FFbDs...XG70Nz5	PHARMACIE	Global	Utilisateur Wallet De: 1000-01-01 gaghaemza haghaem	[ACTIF]
SDqZQwJ...dFTowk13	MEDICIN	Global	Utilisateur Wallet De: 1000-01-01 Chez Ensi ensi	[APPROUVER] [REJETER]
5MA6k1W...770L425C	PATIENT	TANGER-TETOUAN-AL HOCEIMA	Utilisateur Wallet	

FIGURE 4.9 – Tableau de bord Administrateur : Gouvernance et attribution des rôles

4.7 Défis Techniques Rencontrés et Solutions

TABLE 4.2 – Défis techniques et solutions implémentées

Défi	Solution implémentée
Latence IPFS et Rate Limiting	Algorithme de retry exponentiel (<code>withUploadRetry</code> dans <code>ipfsClient.ts</code>) avec fallback vers un CID temporaire <code>pending:*</code> pour ne pas bloquer le flux médical. Proxy Edge API Next.js pour contourner les blocages CORS.
Distribution des clés publiques	Le Smart Contract intègre un registre de clés publiques (<code>encryption_keys</code> Mapping) interrogeable via <code>encryption_key_of</code> . Un mode dégradé (<code>PLAIN_IPFS_FALLBACK</code>) est activé si la clé est manquante.
Coûts de Gas imprévisibles	Le Frontend exécute des Dry Runs (simulations read-only) avant chaque transaction pour estimer le poids (<code>WeightV2</code>) et ajoute un buffer de 20% (<code>txGasLimitFromEstimate</code>).
<code>crypto.subtle</code> undefined	L'API Web Crypto n'est disponible que dans un Secure Context (HTTPS ou localhost). Solution : documentation de déploiement et utilisation de tunnels ngrok pour les tests réseau.
Persistance blockchain	Migration de <code>-tmp</code> (éphémère) vers <code>-d ./blockchain-data</code> (persistant) pour conserver l'état entre les redémarrages du nœud.

4.8 Conclusion

Le système déployé démontre qu'il est possible de construire une application de santé résiliente, entièrement sans serveur backend propriétaire, en déléguant la vérité absolue à la blockchain et le stockage sécurisé au réseau pair-à-pair IPFS. Chaque composant — du chiffrement hybride à la vérification d'intégrité SHA-256 — contribue à un écosystème où la falsification documentaire est mathématiquement impossible.

Conclusion Générale et Perspectives

Le projet « Ma Santé En Chaîne » a constitué un défi d'ingénierie logicielle majeur mobilisant quatre membres de l'équipe pendant plusieurs mois. L'application réussie des principes du Web3 au secteur critique de la santé démontre la viabilité de ces technologies au-delà du seul domaine de la cryptomonnaie.

Au cours de ce travail, nous avons conçu et implémenté un écosystème complet comprenant :

- Un Smart Contract ink! robuste en Rust gérant plus de 15 fonctions publiques (ancrages, claims, ACL, clés de chiffrement).
- Une bibliothèque cryptographique frontend (`medicalCrypto.ts`) implémentant un schéma d'enveloppe hybride AES-GCM + Curve25519.
- Un module de communication blockchain (`chainContract.ts`) avec cache, retry, estimation de Gas dynamique et gestion d'erreurs exhaustive.
- Sept tableaux de bord spécialisés offrant une expérience utilisateur complète pour chaque acteur du système de santé.
- Une architecture « Wallet-First » éliminant totalement le besoin d'un backend centralisé (Node.js/MongoDB).

L'un des acquis majeurs est la preuve qu'un pharmacien ne peut **techniquement** pas valider une ordonnance falsifiée : les mathématiques du hachage SHA-256 combinées à l'immutabilité de la blockchain Substrate l'en empêcheront toujours.

Perspectives d'évolution :

- **Déploiement Mainnet** : Migration vers un réseau Testnet public Polkadot, puis intégration comme Parachain dédiée à la santé.
- **Preuves à Divulcation Nulle (ZKP)** : Intégration de zk-SNARKs permettant à un patient de prouver son éligibilité à un remboursement sans révéler la nature de son acte médical.
- **Intégration IoT Médical** : Permettre aux dispositifs médicaux connectés de posséder leur propre Wallet et d'ancrer directement leurs mesures sur la blockchain.
- **Interopérabilité HL7/FHIR** : Adapter les schémas de données pour être compatibles avec les standards internationaux d'interopérabilité en santé.

En conclusion, ce projet valide notre conviction : l'avenir des données de santé passera par des architectures décentralisées, redonnant aux patients la souveraineté totale sur leurs informations les plus sensibles.

Références

1. Parity Technologies. *Substrate Developer Documentation*. <https://docs.substrate.io/>
2. Parity Technologies. *ink! Smart Contracts Documentation*. <https://use.ink/>
3. Protocol Labs. *IPFS Documentation*. <https://docs.ipfs.tech/>
4. Pinata. *Pinata API Documentation*. <https://docs.pinata.cloud/>
5. Polkadot.js. *Polkadot API Documentation*. <https://polkadot.js.org/docs/>
6. D.J. Bernstein. *Curve25519 : New Diffie-Hellman Speed Records*. PKC 2006.
7. NIST. *Advanced Encryption Standard (AES)*. FIPS PUB 197, 2001.
8. NIST. *Recommendation for Block Cipher Modes of Operation : GCM*. SP 800-38D, 2007.
9. Next.js. *Next.js Documentation*. <https://nextjs.org/docs>
10. TweetNaCl.js. *TweetNaCl.js - A port of TweetNaCl to JavaScript*. <https://tweetnacl.js.org/>